Trying out some exercises

Ex. 1 - Mock Example: Implement the class Stack such that the following test function works:

```
void testStack() {
    Stack<std::string> s{2};
    assert(s.getMaxCapacity() == 2);
    try {
        s = s + "exam";
        s = s + "oop";
        s = s + "fail"; // throws
    }
    catch (std::exception& e) {
        assert(strcmp(e.what(), "Stack is full!") == 0);
    }
    assert(s.pop() == "oop");
    assert(s.pop() == "exam");
}
```

```
template <typename T>
class Stack
{ private:
    int capacity;
    std::vector<T> data;
  public:
    Stack(int maxCapacity);
    ~Stack();
    int getMaxCapacity();
    T pop();
    bool isFull();
    Stack<T>& operator+(const T& elem);
};
```

```
template <typename T>
Stack<T>::Stack(int maxCapacity):capacity(maxCapacity) {}

template <typename T>
Stack<T>::~Stack() = default;

template <typename>
int Stack<T>::getMaxCapacity() {
    return capacity; }

template <typename T>
bool Stack<T>::isFull() {
    return data.size()==capacity; }

template <typename T>
T Stack<T>::pop() {
    if (data.empty()) {
        throw std::runtime_error("Stack is empty!");
    }
    T top = data.back();
    data.pop_back();
    return top;
}

template <typename T>
Stack<T> Stack<T>::operator+(const T& elem) {
    if (isFull()) {
        throw std::runtime_error("Stack is Full");
    }
    data.push_back(elem);
    return *this;
}
```

EXAM GUIDE

Given the test function below, implement the class **Stack**. Specify the method which adds an element to the stack.

similar with the example above

```
void testStack()
{
    Stack<std::string> s{ 2 };
    assert(s.getMaxCapacity() == 2);
    try    {
        s = s + "examination";
        s = s + "oop";
        s = s + "test";
    }
    catch (std::exception& e) {
        assert(strcmp(e.what(), "Stack is full!") == 0);
    }
    assert(s.pop() == "oop");
    assert(s.pop() == "examination");
}
```

Define the classes "ToDo" and "Activity" such that the following C++ code is correct and its results are the ones indicated in the comments.

```
void ToDoList()
{
	ToDo<Activity> todo{};
	Activity tiff{ "go to TIFF movie", "20:00" };
	todo += tiff;
	Activity project{ "present project assignment", "09.20" };
	todo += project;

	// iterates through the activities and prints them as follows:
	// Activity go to TIFF movie will take place at 20.00.
	// Activity present project assignment will take place at 09.20.
	for (auto a : todo)
		std::cout << a << '\n';

	// Prints the activities as follows:
	// Activity present project assignment will take place at 09.20.
	// Activity go to TIFF movie will take place at 20.00.
	todo.reversePrint(std::cout);
}
```

```
class Activity
{ private:
    std::string desculption;
    std::string startTime;
  public:
    Activity(std::string d, std::string t);
    friend std::ostream &operator<<(std::ostream
                    &os, const Activity & act);
};

template <typename T>
class ToDo
{ private:
    std::vector<T> activities;
  public:
    ToDo() = default;
    ToDo<T>& operator+=(const T& activity);
    void reverseBuint(std::ostream &os);
};
```

```
Activity::Activity(std::string & d, std::string &t):
    desciption(d), startTime(t) {}
std::ostream& operator<<(std::ostream os,
    Activity(act) {
  os<<"Activity" << act.desciption << "will take
    place at "<<act.startTime<<".";
  return os;
}
template<typename T>
ToDo<T>& ToDo<T>::operator+=(const T& activity)
{  activities.push_back(activity);
   return *this;
}
template<typename T>
void ToDo<T>::reverseBuint(std::ostream &os)
{  for(int i=activities.size()-1; i>=0; --i)
     os<<activities[i]<<'\n';
}
```

Determine the result of the execution of the following C++ programs. If there are any errors, signal them, correct them and explain the result after the error has been corrected. Justify your answers.

```
class B
{
public:
	B() { std::cout << "B{}"; }
	virtual void print() { std::cout <<
"B"; }
	virtual ~B() { std::cout << "~B()"; }
};

class D : public B
{
public:
	D() { std::cout << "D{}"; }
	void print() override { std::cout <<
"D"; }
	virtual ~D() { std::cout << "~D()"; }
};
```

B{} B{}D{}

```
int main()
{
	B* b[] = { new B{}, new D{} };
	b[0]->print(); → B
	b[1]->print(); → D
	delete b[0];   → ~B()
	delete b[1];   → ~D() ~B()
	return 0;
}
```

Result:

B{}B{}D{}B~B()~D()~B()

```cpp
class Person
{
public:
	Person() { std::cout << "Person{}"; }
	virtual void print() = 0;
	virtual ~Person() { std::cout << "~Person()"; }
};

class Student : public Person
{
public:
	Student() { std::cout << "Student{}";
}
	void print() override { std::cout << "Student"; }
	virtual ~Student() { std::cout << "~Student()"; }
};
```

it tells the compiler "it must be overridden in a derived class"

Person is an abstract class (since we have at least a function pure virtual =0) so it cannot be instantiated directly. It's designed to be a base class for another class.

```cpp
int main()
{
	Person* p = new Person{};
	delete p;
	Person* s = new Student{};
	s->print();
	delete s;

	return 0;
}
```

delete these 2 lines; why?

Person{} Student{}

Student

~Student() ~Person()

Result: Person{} Student{} Student ~Student() ~Person()

```cpp
class E
{
public:
	E() { std::cout << "E{}"; }
	virtual ~E() { std::cout << "~E()"; }
};

class DE : public E
{
public:
	static int n;
	DE() { n++; }
};

int DE::n = 0;


int fct2(int x)
{
	if (x < 0)
	{
		throw E{};
		std::cout << "number less than 0" << std::endl;
	}
	else if (x == 0)
	{
		throw DE{};
		std::cout << "number equal to 0" << std::endl;
	}
	return x % 10;
}
```

unreachable

unreachable

```cpp
int main()
{
	try
	{
		int res = 0;
		res = fct2(-5);
		std::cout << DE::n;
		res = fct2(0);
		std::cout << DE::n;
		res = fct2(25);
		std::cout << DE::n;
	}
	catch (E&)
	{
		std::cout << DE::n;
	}

	return 0;
}
```

E{}

throw ⇒ stop

these doesn't execute anymore

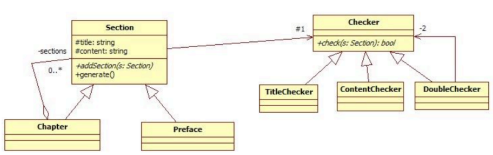
0

+ destructor → E()

Result: E{}0E()

5.2. Inheritance, polymorphism, UML, design patterns – C++

1. Write a C++ application for generating a book when several chapters are given **(4p)**, as follows:
   a. The class **Checker** is abstract, it contains the pure virtual function *check*, which checks if the given section is correct. **(0.2p)**
   b. The classes **TitleChecker** and **ContentChecker** check the title and the content of the given section, respectively. The function *check* in **TitleChecker** returns true is the section title is longer than 2 characters. The function *check* in **ContentChecker** returns true if each sentence in the content begins with a capital letter (hint: function *isupper(char)*). **(0.5p)**
   c. Create the class **DoubleChecker**. Its function *check* returns true only if the section is correct (check returns true), from the point of view of both aggregated checkers. **(0.5p)**
   d. Create the class **Section**, with abstract function *addSection*. The function *generate* prints the title and content of the section only if the section is correct from the point of view of the aggregated checker. **(0.7p)**
   e. Create the class **Preface**. A preface cannot have any aggregated sections. **(0.3p)**
   f. Create the class **Chapter**. The function generate in the class **Chapter** will generate the chapter only if its title and content are correct and will include only those aggregated sections that are correct (checked). **(0.7p)**
   g. Generate a book, which contains: a preface (checked by content, with a content checker); a chapter (checked by title, with a title checker), which contains 2 subchapters, both checked by title and by content; the first chapter passes the check, while the second does not. Make sure the memory is correctly managed. **(1.1p)**



```
class Checker
{ public:
      virtual bool check(Section& s) = 0;
};
class TitleChecker: class Checker
{ public:
      bool check( Section& s) override;
};
class Content_Checker : class Checker
{ public:
      bool check ( Section& s) override;
};
class DoubleChecker : class Checker
{ public:
      bool check(Section& s) override;
};
class Section
{ private:
      std::string title;
      std:: string content;
  public:
     virtual void addSection(Section& s) = 0;
     virtual void generate();
};
class Preface: class Section
{ public:
      void addSection(Section& s) override;
      void generate() override;
};
class Chapter : class Section
{ private:
     std::vector<Section&> sections;
  public:
      void addSection(Section& s) override;
      void generate() override;
};
int main()
{  ContentChecker contentChecker;
   Section* preface = new Preface("intro", "This is the preface", &contentChecker);

 etc.
```

```
bool TitleChecker::check(Section& s)
{    return (s.title.size >=2)==1;
}
```

Trying out some subjects

WRITTEN EXAMINATION – ROW 1

1. Define the classes **Object, Integer, String** and **MyObjectList** such that the following C++ code is correct and memory is correctly managed. Enumerate the data members (including types) and the prototypes of all methods and implement the method allowing the operation emphasized in the code below. **(1.4p)**

```
void function()
{
	MyObjectList list{};
	list.add(new Integer{ 2 }).add(new String{ "Hi" });
	String* s = new String{ "Bye" };
	assert(*s == "Bye");
	list.add(s).add(new Integer{ 5 }); // IMPLEMENT THIS OPERATION
	assert(list.length() == 4);

	// prints: 2 Hi Bye 5
	for (Object* o : list)
		o->print();
}
```

2. Determine the result of the execution of the following C++ programs. **Justify your answers. (4x0.9p)**

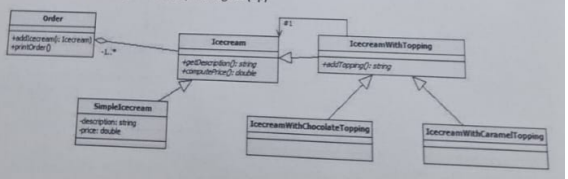
```
// a)
class Base {
private:
	virtual void f() { std::cout << "Base" << "\n"; }
public:
	virtual ~Base() { f(); }
	void g() { f(); }
};

class Derived : public Base {
private:
	void f() { std::cout << "Derived" << "\n"; }
public:
	void h() { Base::g(); cout << "Function h" << "\n"; }
	~Derived() { f(); }
};

int main() {
	Base* b = new Derived{};
	b->g();
	Derived *d = dynamic_cast<Derived*>(b);
	if (d != nullptr)
		d->h();
	delete b;
	return 0;
}
```

```
// b)
class Exception1 {
public:
	Exception1() { cout << "Exception1 "; }
	virtual void fct() { cout << "Ex1 "; }
	virtual ~Exception1() { cout << "~Exception1 "; }
};

class Exception2 : public Exception1 {
public:
	void fct() override { cout << "Ex2 "; }
	virtual ~Exception2() { cout << "~Exception2 "; }
};

string f(int x) {
	cout << "Hi ";
	if (x % 2 == 0)
		throw Exception1{};
	else
		throw Exception2{};
	return string{ "Bye " };
}

int main() {
	try {
		cout << f(5);
		cout << f(2);
	}
	catch (Exception1& ex) { ex.fct(); }
	catch (Exception2& ex2) { ex2.fct(); }
	return 0;
}
```

```
// c)
class A {
private:
	int x;
public:
	A(int _x = 0) : x{ _x } {}
	int get() const { return x; }
	friend class B;
};

class B {
private:
	A a;
public:
	B(const A& _a) : a{ _a } {}
	B& operator+(const A& _a) {
		a.x += _a.x;
		return *this;
	}
	int get() { return a.x; }
};

int main() {
	A a1{ 1 }, a2{ 2 };
	cout << a1.get() << " " << a2.get() << " ";

	B b1{ a1 };
	B b2 = b1 + a2 + a1;
	cout << b1.get() << " " << b2.get();
	return 0;
}
```

```
// d)
int main() {
	vector<string> v{ "rain", "in", "spain", "falls", "mainly", "plain" };
	sort(v.begin(), v.end(), [](string a, string b) { return a > b; });
	vector<string>::iterator it = v.begin() + 4;
	*it = "brain";
	it = v.begin();
	while (it != v.end())
	{
		string aux = *it;
		char c1 = aux[aux.size() - 3];
		char c2 = aux[aux.size() - 2];
		char c3 = aux[aux.size() - 1];
		if (c1 == 'a' && c2 == 'i' && c3 == 'n')
			cout << *it << "\n";
		it++;
	}
	return 0;
}
```

3. Write a C++ application for ordering ice cream **(4p)**, as follows:
   a. The class **Icecream** is abstract, it contains the abstract functions *getDescription()* and *computePrice()*. **(0.3p)**
   b. The class **SimpleIcecream** contains a description and a price. The method *getDescription()* returns the concatenation of the string "simple ice cream with" with the ice cream's description. The method *computePrice()* returns the ice cream's price. **(0.5p)**
   c. The class **IcecreamWithTopping** contains the pure virtual function *addTopping()*. The description of an ice cream with topping will be the description of the aggregated ice cream concatenated to the type of topping returned by the *addTopping()* function. **(0.7p)**
   d. The function *addTopping()* in the classes **IcecreamWithChocolateTopping** and **IcecreamWithCaramelTopping** returns "with chocolate topping" and "with caramel topping", respectively. The price of an ice cream with chocolate topping increases by 3 RON, while the price of an ice cream with a caramel topping increases by 2 RON. **(0.8p)**
   e. The class **Order** allows the creation of an order containing several ice creams. The method *printOrder()* prints the description and the price of each ice cream, in descending order of the prices. **(0.7p)**
   f. Write a function which creates an order containing: a simple vanilla ice cream, a pistachio ice cream with chocolate and caramel topping, a chocolate ice cream with caramel topping and a simple hazelnuts ice cream. Make sure the memory is correctly managed. **(1p)**



- of

WRITTEN EXAMINATION – ROW 1

1. Define the classes **SmartPointer** and **Vector** such that the following C++ code is correct. Enumerate the data members (including types) and the prototypes of all methods and implement the method allowing the operation emphasized in the code below. **(2p)**

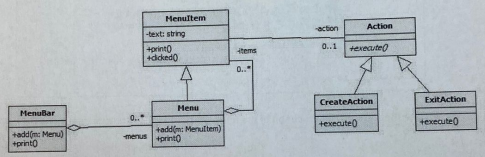
```
void function1() {
	SmartPointer<int> i1{ new int{ 1 } };
	SmartPointer<int> i2{ new int{ 2 } };
	SmartPointer<int> i3{ new int{ 3 } };
	Vector<SmartPointer<int>> v1{};
	v1.add(i1).add(i2).add(i3);
	for (auto e : v1)
		cout << *e << ", "; // prints 1, 2, 3

	SmartPointer<string> s1{ new string{ "A" } };
	SmartPointer<string> s2 = s1;
	SmartPointer<string> s3{ new string{ "C" } };
	Vector<SmartPointer<string>> v2{};
	v2.add(s2).add(s1);
	try {
		v2 = v2 - s2;  // IMPLEMENT THIS OPERATION
		v2 = v2 - s3;  // IMPLEMENT THIS OPERATION
	}
	catch (std::runtime_error& ex) {
		cout << ex.what(); // prints: "Element does not exist!"
	}
} // memory is correctly deallocated
```

2. Determine the result of the execution of the following C++ programs. **Justify your answers. (2 x 1.5p)**

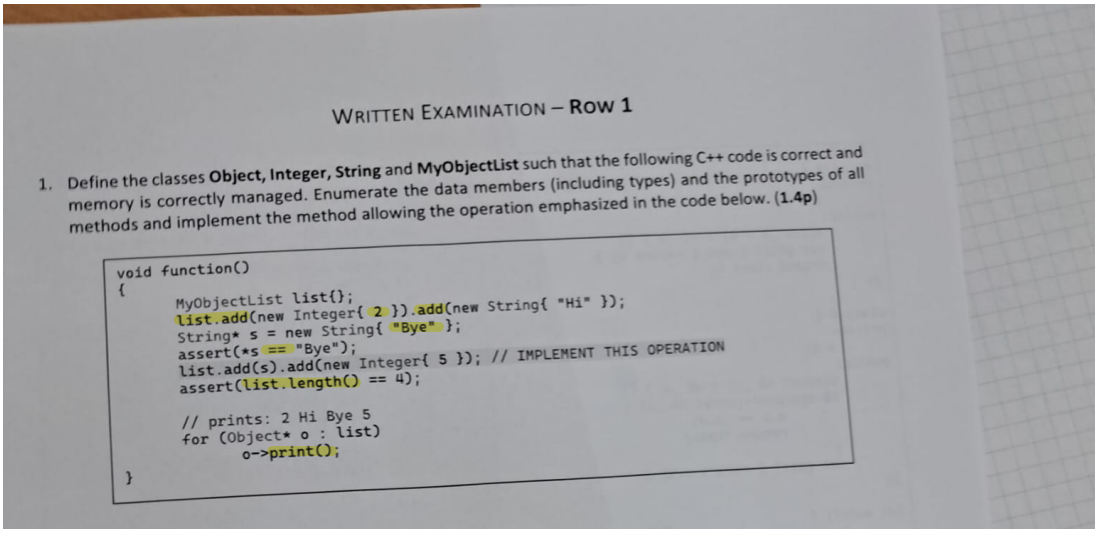
```
// a)
class A {
private:
	int* x;

public:
	static int noOfInstances;

	A(int _x = 0) {
		x = new int{ _x };
		noOfInstances++;
	}
	int get() { return *x; }
	void set(int _x) { *x = _x; }
	~A() { delete x; }
};

int A::noOfInstances = 0;

int main()
{
	A a1, a2;
	cout << a1.noOfInstances << " ";
	A a3 = a1;
	cout << A::noOfInstances << " ";
	a1.set(8);
	cout << a1.get() << " ";
	cout << a2.get() << " ";
	cout << a3.get() << " ";

	return 0;
}
```

```
// b)
class B {
public:
	B() {}
	B(const B&) { cout << "copy "; }
	virtual B f() { cout << "B.f "; return *this; }
	virtual ~B() { cout << "~B "; }
};

class D : public B
{
private:
	B* b;
public:
	D(B* _b) : b{ _b } { cout << "D "; }
	B f() override { cout << "D.f "; return b->f(); }
};

int main()
{
	B* b = new B();
	B* d = new D{ b };
	d->f();
	delete d;
	delete b;
	return 0;
}
```

3. Write a C++ application which demonstrates the construction of menus, as follows **(4p)**:
   a. The class *Action* is abstract. The *execute* function will print "Create file" for a *CreateAction* and "Exit application" for an *ExitAction*. **(0.7p)**
   b. A *MenuItem* has a text and an associated action. When a menu item is displayed (function *print()*), its text is displayed. When a *MenuItem* is clicked (function *clicked()*), its text is displayed and its associated action (if this is valid), is executed. **(1p)**
   c. A *Menu* can contain several menu items. However, a menu can also be a menu item for another menu (a submenu). When a menu is displayed, besides showing the menu's text, all its submenus are displayed. **(0.8p)**
   d. A *MenuBar* contains several *Menus*. When a menu bar is displayed, all its elements are displayed. **(0.5p)**
   e. The main application will show a menu bar containing the menus *File* and *About*. None of these have any actions associated. *File* has *New* and *Exit* as submenus and New has *Text* and *C++* as submenus. Exit has an *ExitAction* associated, while *Text* and *C++* each have a *CreateAction*. Simulate the clicking of the following sequence: File -> New -> C++ and then Exit and show the actions that are executed. Take memory management into consideration and implement it correctly. **(1p)**



1p - of



WRITTEN EXAMINATION – ROW 1

1. Define the classes **Object, Integer, String** and **MyObjectList** such that the following C++ code is correct and memory is correctly managed. Enumerate the data members (including types) and the prototypes of all methods and implement the method allowing the operation emphasized in the code below. (1.4p)

```
void function()
{
        MyObjectList list{};
        list.add(new Integer{ 2 }).add(new String{ "Hi" });
        String* s = new String{ "Bye" };
        assert(*s == "Bye");
        list.add(s).add(new Integer{ 5 }); // IMPLEMENT THIS OPERATION
        assert(list.length() == 4);

        // prints: 2 Hi Bye 5
        for (Object* o : list)
                o->print();
}
```

```
class Object
{ public:   Object() = default;
            virtual ~Object() = default;
            virtual void print() const = 0;
};
class Integer : public Object
{ private: int number;
  public:  Integer(int number);
           void print() const override;
};
class String: public Object
{ private: std::string text;
  public:  String(std::string text);
           void print() const override;
           bool operator==(const std::
                string & other) const;
};
class MyObjectList
{ private: std::vector<Object*> objects; );
  public:  MyObjectList& add(Object* obj);
           int length();
           ~MyObjectList();
           auto begin() const;
           auto end() const;
};
```

```
void Integer::print() const
{   std::cout << number << " ";
}
void String::print() const
{ std::cout << text << " ";
}
bool String::operator==(const std::string & other) const
{   return text == other;
}
int ObjectList::length()
{   return objects.size());
}
MyObjectList & MyObjectList::add( Object* obj)
{   objects.push_back(obj);
    return *this;
}
auto MyObjectList::begin() const
{ return objects.begin();}
auto MyObjectList::end() const
{ return objects.end();}
MyObjectList::~MyObjectList()
{   for( auto obj: objects)
        delete obj;
}
```

---

some rules:

"new" → make sure there's a destructor that deletes each object

use "const" for getters, print, check or any logic that doesn't modify the object

use a virtual destructor in base classes

"return *this" when chaining

2. Determine the result of the execution of the following C++ programs. **Justify your answers. (4x0.9p)**

```
// a)
class Base {
private:
	virtual void f() { std::cout << "Base" << "\n"; }
public:
	virtual ~Base() { f(); }
	void g() { f(); }
};

class Derived : public Base {
private:
	void f() { std::cout << "Derived" << "\n"; }
public:
	void h() { Base::g(); cout << "Function h" << "\n"; }
	~Derived() { f(); }
};

int main() {
	Base* b = new Derived{};
	b->g();
	Derived *d = dynamic_cast<Derived*>(b);
	if (d != nullptr)
		d->h();
	delete b;
	return 0;
}
```

```
// b)
class Exception1 {
public:
	Exception1() { cout << "Exception1 "; }
	virtual void fct() { cout << "Ex1 "; }
	virtual ~Exception1() { cout << "~Exception1 "; }
};

class Exception2 : public Exception1 {
public:
	void fct() override { cout << "Ex2 "; }
	virtual ~Exception2() { cout << "~Exception2 "; }
};

string f(int x) {
	cout << "Hi ";
	if (x % 2 == 0)
		throw Exception1{};
	else
		throw Exception2{};
	return string{ "Bye " };
}

int main() {
	try {
		cout << f(5);
		cout << f(2);
	}
	catch (Exception1& ex) { ex.fct(); }
	catch (Exception2& ex2) { ex2.fct(); }
	return 0;
}
```

Derived Derived Function h Derived Base
\n

Hi Exception1 Ex 2 ~Exception2 ~Exception1

```
// c)
class A {
private:
	int x;
public:
	A(int _x = 0) : x{ _x } {}
	int get() const { return x; }
	friend class B;
};

class B {
private:
	A a;
public:
	B(const A& _a) : a{ _a } {}
	B& operator+(const A& _a) {
		a.x += _a.x;
		return *this;
	}
	int get() { return a.x; }
};

int main() {
	A a1{ 1 }, a2{ 2 };
	cout << a1.get() << " " << a2.get() << "\n";
	B b1{ a1 };
	B b2 = b1 + a2 + a1;
	cout << b1.get() << " " << b2.get();
	return 0;
}
```

1 2

$b_1.a.x = 1$

$b_1 + a_2 \Rightarrow b_1.x += a_2.x \rightarrow b_1.x = 1+2 = 3$

$+ a_1.x \rightarrow b_1.x = 3+1 = 4$

$\Rightarrow b_2.x = 4$

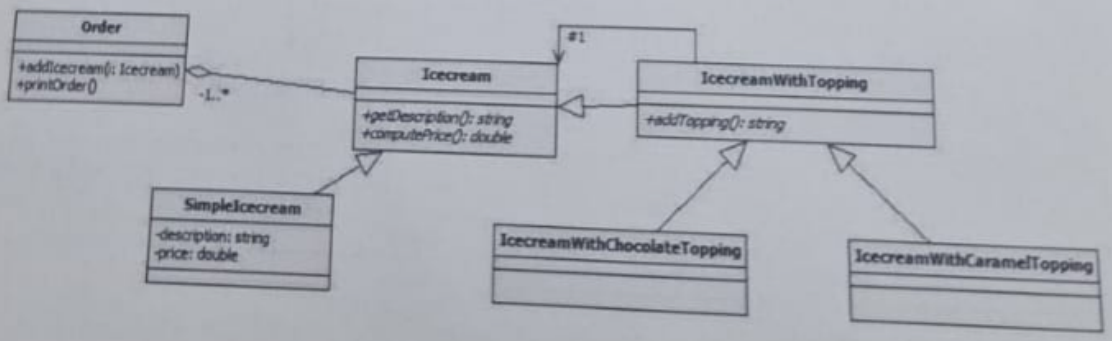
4 4

```
// d)
int main() {
	vector<string> v{ "rain", "in", "spain", "falls", "mainly", "plain" };
	sort(v.begin(), v.end(), [](string a, string b) { return a > b; });
	vector<string>::iterator it = v.begin() + 4;
	*it = "brain";
	it = v.begin();
	while (it != v.end())
	{
		string aux = *it;
		char c1 = aux[aux.size() - 3];
		char c2 = aux[aux.size() - 2];
		char c3 = aux[aux.size() - 1];
		if (c1 == 'a' && c2 == 'i' && c3 == 'n')
			cout << *it << "\n";
		it++;
	}
	return 0;
}
```

spain rain plain ~~mainly~~ brain in falls

spain rain plain brain

3. Write a C++ application for ordering ice cream **(4p)**, as follows:
   a. The class **Icecream** is abstract, it contains the abstract functions *getDescription()* and *computePrice()*. **(0.3p)**
   b. The class **SimpleIcecream** contains a description and a price. The method *getDescription()* returns the concatenation of the string "simple ice cream with" with the ice cream's description. The method *computePrice()* returns the ice cream's price. **(0.5p)**
   c. The class **IcecreamWithTopping** contains the pure virtual function *addTopping()*. The description of an ice cream with topping will be the description of the aggregated ice cream concatenated to the type of topping returned by the *addTopping()* function. **(0.7p)**
   d. The function *addTopping()* in the classes **IcecreamWithChocolateTopping** and **IcecreamWithCaramelTopping** returns "with chocolate topping" and "with caramel topping", respectively. The price of an ice cream with chocolate topping increases by 3 RON, while the price of an ice cream with a caramel topping increases by 2 RON. **(0.8p)**
   e. The class **Order** allows the creation of an order containing several ice creams. The method *printOrder()* prints the description and the price of each ice cream, in descending order of the prices. **(0.7p)**
   f. Write a function which creates an order containing: a simple vanilla ice cream, a pistachio ice cream with chocolate and caramel topping, a chocolate ice cream with caramel topping and a simple hazelnuts ice cream. Make sure the memory is correctly managed. **(1p)**



- of

```
class Icecream
{ public: virtual std::string getDescription() const=0
          virtual double computePrice() =0;
          virtual ~Icecream() = default;
          virtual Icecream() = default;
};
class SimpleIcecream: public Icecream
{ private: string description;
          double price;
  public: std::string getDescription() const override;
          double computePrice() override;
          SimpleIcecream(std::string & desc, double p);
};
class IcecreamWithTopping: public Icecream
{ protected: Icecream * base;
  public: IcecreamWithTopping(Icecream * baseCream);
          virtual std::string addTopping() const = 0;
          std::string getDescription() const override;
          virtual ~IcecreamWithTopping();
};
class IcecreamWithChocolateTopping: public IcecreamWithTopping
{ public:
    IcecreamWithChocolateTopping(Icecream * baseCream): IcecreamWithTopping(baseCream) {}
    std::string addTopping() const override { return "with chocolate topping"; }
    double computePrice() const override { return base->computePrice() + 3; }
};
```

```
SimpleIcecream::SimpleIcecream(std::string & desc, double p): description(desc), price(p) {}
string SimpleIcecream::getDescription() const
{ return "simple ice cream with " + description; }
double SimpleIcecream::computePrice()
{ return price; }

IcecreamWithTopping::IcecreamWithTopping(Icecream * baseCream): base(baseCream) {}
std::string IcecreamWithTopping::getDescription() const
{ return base->getDescription() + " " + addTopping(); }
IcecreamWithTopping::~IcecreamWithTopping()
{ delete base; }
```

```
class IcecreamWithCaramelTopping : public IcecreamWithTopping
{
public:
    IcecreamWithCaramelTopping(Icecream* baseCream) : IcecreamWithTopping(baseCream) {}
    std::string addTopping() const override { return "with caramel topping"; }
    double computePrice() const override { return base->computePrice() + 2; }
};

class Order
{
private: std::vector<Icecream*> icecreams;
public: void addIcecream(IceCream* i) { icecreams.push_back(i); }
        void printOrder() { std::vector<IceCream*> sorted = icecreams;
                            std::sort(sorted.begin(), sorted.end(), [](Icecream *a, Icecream *b) {
                                return a->computePrice() > b->computePrice(); });
                            for(auto i: sorted)
                                std::cout<< i->getDescription() << "_" << i->computePrice() << " RON\n";
                          }
        ~Order() { for(auto i: icecreams) delete i; }
};

int main()
{
    Order order;
    Order.addIcecream(new SimpleIceCream("vanilla", 5));
    Icecream* p = new SimpleIcecream("pistachio", 6);
    p = new IcecreamWithChocolateTopping(p);
    p = new IcecreamWithCaramelTopping(p);
    order.addIcecream(p);
    Icecream* c = new SimpleIcecream("chocolate", 4.5);
    c = new IcecreamWithCaramelTopping(c);
    order.add(c);
    order.add(new SimpleIcecream("hazelnuts", 5.5));
    order.printOrder();
    return 0;
}
```

WRITTEN EXAMINATION – ROW 1

1. Define the classes **SmartPointer** and **Vector** such that the following C++ code is correct. Enumerate the data members (including types) and the prototypes of all methods and implement the method allowing the operation emphasized in the code below. **(2p)**

```
void function1() {
      SmartPointer<int> i1{ new int{ 1 } };
      SmartPointer<int> i2{ new int{ 2 } };
      SmartPointer<int> i3{ new int{ 3 } };
      Vector<SmartPointer<int>> v1{};
      v1.add(i1).add(i2).add(i3);
      for (auto e : v1)
            cout << *e << ", "; // prints 1, 2, 3

      SmartPointer<string> s1{ new string{ "A" } };
      SmartPointer<string> s2 = s1;
      SmartPointer<string> s3{ new string{ "C" } };
      Vector<SmartPointer<string>> v2{};
      v2.add(s2).add(s1);
      try {
            v2 = v2 - s2;  // IMPLEMENT THIS OPERATION
            v2 = v2 - s3;  // IMPLEMENT THIS OPERATION
      }
      catch (std::runtime_error& ex) {
            cout << ex.what(); // prints: "Element does not exist!"
      }
} // memory is correctly deallocated
```

constructor 1 ← SmartPointer<int> i1{ new int{ 1 } };
constructor 2 ← Vector<SmartPointer<int>> v1{};
// pointers
return *this

```
template <typename T>
class SmartPointer
{ private: T* elem;
  public: SmartPointer( T* elem = nullptr);
          SmartPointer (const SmartPointer <T>& other);   // copy constructor ; without this, both SmartPoint. would point to the same memory
          SmartPointer<T>& operator =(const SmartPointer<T>& other);   => double delete & crash (in destructor)
          ~SmartPointer ();

          T& operator*() const;   // dereference operator -> allows to use smart pointer like a normal pointer
          bool operator ==(const SmartPointer<T>& other) const;
};
```

=> copy assignment operator

The rule of 3: if a class manages dynamic memory:
- copy constructor
- copy assignment operator
- destructor

missing any of this => memory bugs, leaks & crashes

```
template <typename T>
SmartPointer<T>::SmartPointer(T* p) : ptr(p) {}

template <typename T>
SmartPointer<T>::SmartPointer(const SmartPointer<T>& other) {
    if (other.ptr)
        ptr = new T(*other.ptr);
    else
        ptr = nullptr;
}

template <typename T>
SmartPointer<T>& SmartPointer<T>::operator=(const SmartPointer<T>& other) {
    if (this != &other) {
        delete ptr;
        ptr = (other.ptr) ? new T(*other.ptr) : nullptr;
    }
    return *this;
}

template <typename T>
SmartPointer<T>::~SmartPointer() {
    delete ptr;
}

template <typename T>
T& SmartPointer<T>::operator*() const {
    return *ptr;
}

template <typename T>
bool SmartPointer<T>::operator==(const SmartPointer<T>& other) const {
    return ptr && other.ptr && *ptr == *other.ptr;
}
```

```
template <typename T>
class Vector {
private:
    std::vector<T> data;

public:
    Vector<T>& add(const T& element);
    Vector<T> operator-(const T& element) const;

    typename std::vector<T>::iterator begin();
    typename std::vector<T>::iterator end();
};

template <typename T>
Vector<T>& Vector<T>::add(const T& element) {
    data.push_back(element);
    return *this;
}

template <typename T>
Vector<T> Vector<T>::operator-(const T& element) const {
    Vector<T> result;
    bool found = false;

    for (const auto& e : data) {
        if (!(e == element)) {
            result.data.push_back(e);
        } else {
            found = true;
        }
    }

    if (!found)
        throw std::runtime_error("Element does not exist!");

    return result;
}

template <typename T>
typename std::vector<T>::iterator Vector<T>::begin() {
    return data.begin();
}

template <typename T>
typename std::vector<T>::iterator Vector<T>::end() {
    return data.end();
}
```

EXERCISE 1

Practice - generated by GPT

```
int main() {
    Item i1("apple", 5);
    Item i2("banana", 3);
    Item i3("apple", 5);

    Container c;
    c.add(i1).add(i2);

    try {
        c = c - i3;
        c = c - i3;  // should throw
    } catch (std::exception& e) {
        std::cout << e.what() << "\n";
    }

    for (auto i : c)
        std::cout << i << "\n";

    return 0;
}
```

```
class Item
{ private: std::string key;
           int value;
  public: Item(std::string k, int v): key(k), value(v){}
          friend std::ostream & operator<< (std::ostream& cout
            const Item & i)
          bool operator==(const Item& i) const ;
};

class Container
{ private: std::vector<Item> data;
  public: Container() = default;
          Container& add(const Item& i);
          Container operator-(const Item& i) const;
          auto begin() { return data.begin(); }
          auto end() { return data.end(); }
};
```

```
std::ostream & Item::operator<<(std::ostream& cout, const Item &i) const
{   cout << "key" << i.key << "value" << i.value << "\n";
    return cout;
}

bool Item::operator==(const Item &i) const
{   return key == i.key && value == i.value;
}

Container& Container::add(const Item& i)
{   data.push_back(i);
    return *this);
}

Container Container::operator-(const Item& i) const
{   Container result; bool found=false;
    for(auto& e: data)
        if(!(e==i)) result.push_back(e);
        else found=true;
    if(found==false) throw std::runtime_error("Element does not exist");
    return result;
}
```

Some theory

Dynamic (if I see "new", "delete", raw pointers etc.)
- constructor
- copy constr. } rule of 3
- destr. }
- assignment operator

Non-Dynamic - constructor

if it doesn't have any attributes, the constr. is default

on top of that, we need operators
- examples: ==
  +=
  -
  << → usually it needs to be a friend class
    → if we are printing objects (that we created earlier)

```
* for(auto item:container)        <- we need begin/end
      cout << item;
          -> we need
* for(auto item: container)
      cout << "exists";
          -> we DON'T need
* Item i("apple", 5)
      cout << i.getKey();
          -> we DON'T need
```

we also might need begin/end → for classes where we store multiple objects
→ that one most the STL container (like vector)

```
* std::vector<Item>v;
  for(auto x:v)      -> we DON'T need
  ...
```

also, we might need to define operations : add → if it's a chaining op., it needs to have return * this

same for operator +

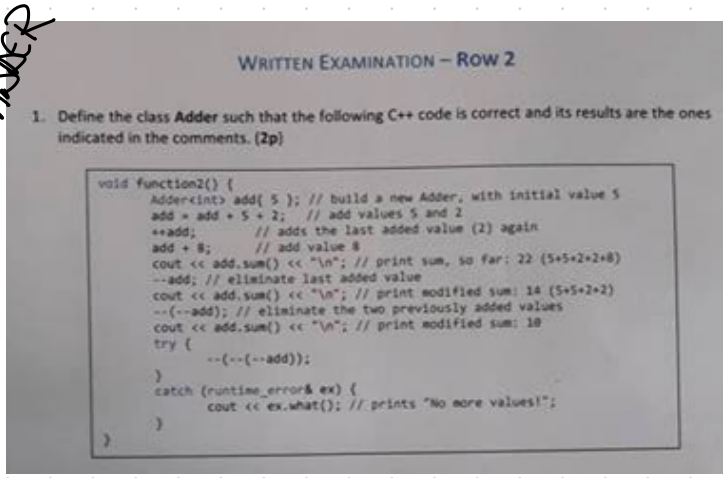
remove/operator - → uses operator == (so we need to define it)

* templates :

```
template <typename T>
class MyClass
{ ...
};
```

! all methods must be prefixed with template<typename T>
! MyClass<T>
→ must specify the type : int, std::string etc)
also, don't forget about defining operators
it basically acts the same as a normal class

ADDER



WRITTEN EXAMINATION – ROW 2

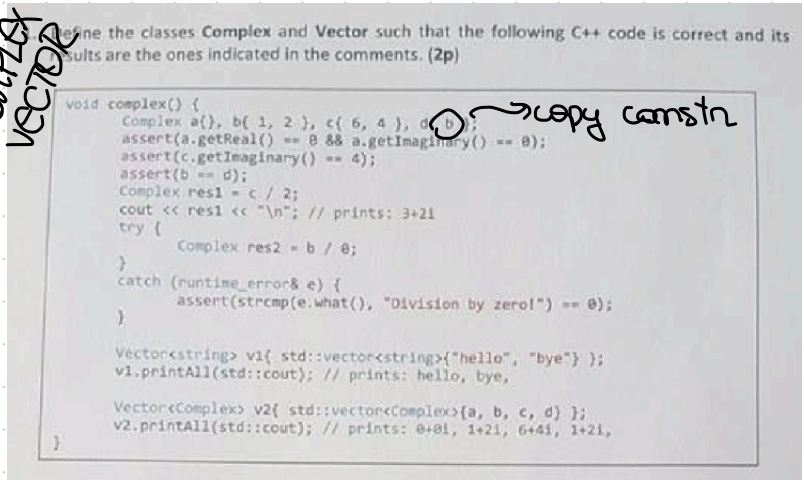
1. Define the class **Adder** such that the following C++ code is correct and its results are the ones indicated in the comments. (2p)

```
void function2() {
        Adder<int> add{ 5 }; // build a new Adder, with initial value 5
        add = add + 5 + 2;   // add values 5 and 2
        ++add;               // adds the last added value (2) again
        add + 8;             // add value 8
        cout << add.sum() << "\n"; // print sum, so far: 22 (5+5+2+2+8)
        --add; // eliminate last added value
        cout << add.sum() << "\n"; // print modified sum: 14 (5+5+2+2)
        --(--add); // eliminate the two previously added values
        cout << add.sum() << "\n"; // print modified sum: 10
        try {
                --(--(--add));
        }
        catch (runtime_error& ex) {
                cout << ex.what(); // prints "No more values!";
        }
}
```

template Adder
- → value: vector<T>
- → constr
- → op+
- → op--
- → sum()
- → op++

```
template <typename T>
class Adder
{   private:  vector<T> data;
    public:   Adder(vector<T> v): data(v) {}
              Adder& operator+(const T& val)
                {   data.push_back(val);
                    return *this;
                }
              Adder& operator++()
                {   data.push_back(data.back);
                    return *this;
                }
              Adder& operator--()
                {   if(data.empty) throw std::runtime_error("No more values");
                    data.pop_back();
                    return *this;
                }
              T sum()
              {  T total = T{};
                 for(auto elem: data)
                    total = total + it;
                 return total;
              }
};
```

COMPLEX VECTOR



Define the classes **Complex** and **Vector** such that the following C++ code is correct and its results are the ones indicated in the comments. (2p)

```
void complex() {
        Complex a{}, b{ 1, 2 }, c{ 6, 4 }, d{ 7, 6 };
        assert(a.getReal() == 0 && a.getImaginary() == 0);
        assert(c.getImaginary() == 4);
        assert(b == d);
        Complex res1 = c / 2;
        cout << res1 << "\n"; // prints: 3+2i
        try {
                Complex res2 = b / 0;
        }
        catch (runtime_error& e) {
                assert(strcmp(e.what(), "Division by zero!") == 0);
        }

        Vector<string> v1{ std::vector<string>{"hello", "bye"} };
        v1.printAll(std::cout); // prints: hello, bye,

        Vector<Complex> v2{ std::vector<Complex>{a, b, c, d} };
        v2.printAll(std::cout); // prints: 0+0i, 1+2i, 6+4i, 1+2i,
}
```

→ copy constr

Complex → int real = 0
int im = 0
- → getReal()
- → getImaginary()
- → op/
- → constr.
- → op==
- → op<<
- → copy constr

```
class Complex
{ private: int real, imaginary;
  public: Complex(int r, int i): real(r), imaginary(i) {}
          Complex(const Complex& other): real(other.real)
                                         imaginary(other.imaginary) {}
          int getReal() {return real;}
          int getImaginary() {return imaginary;}
          bool operator==(const Complex& other) const
            {return real==other.real && imaginary==other.imaginary;}
          Complex operator/(int divisor)
            {  if(divisor==0) throw std::runtime_error("Division
                                                        by zero");
               return Complex(real/divisor, imaginary/divisor);
            }
          friend std::ostream& operator<<(ostream& os, Complex& c)
            {  os<< c.real<< "+"<<c.imaginary<< "i";
               return out;
            }
};
```

Vector (template)
- → vector<T>
- → printAll
- → constr

```
template <typename T>
class Vector
{private: std::vector<T> data;
 public: Vector(const std::vector<T>& d): data(d) {}
         void printAll(std::ostream& out) const
           {   for(auto& elem: data)
                 out<< elem<< ";";
           }
};
```

HTML

Define the **HTMLElement, HTMLImage, HTMLParagraph** and **HTMLBuilder** classes such that the following C++ code is correct, all asserts pass and its results are the ones indicated in the comments. Provide specification for the *getHTMLString()* method.

```
void fct1() {
 HTMLElement* p1 = new HTMLParagraph{ "Examination" };
 assert(p1->getHTMLString() == "<p>Examination</p>");
 HTMLElement* p2 = nullptr;
 HTMLElement* i1 = new HTMLImage{ "a.jpg" };
 HTMLElement* i2 = new HTMLImage{ "b.jpg" };
 assert(i2->getHTMLString() == "<img>b.jpg</img>");
 HTMLBuilder<HTMLElement*> html{};
 try {
  html += p2;
  assert(false);
 } catch (runtime_error& e) {
  assert(strcmp(e.what(), "Cannot add a null element!") == 0); }

 ((html += p1) += i1) += i2;
 cout << html; // prints: <html><body><p>Examination</p><img>a.jpg</img><img>b.jpg</img></html></body>
 delete p1; delete p2;
 delete i1; delete i2;
}
```

HTMLElement → getHTMLString() = 0

image → inh. Element
→ string source
→ constr.
→ getHTMLString

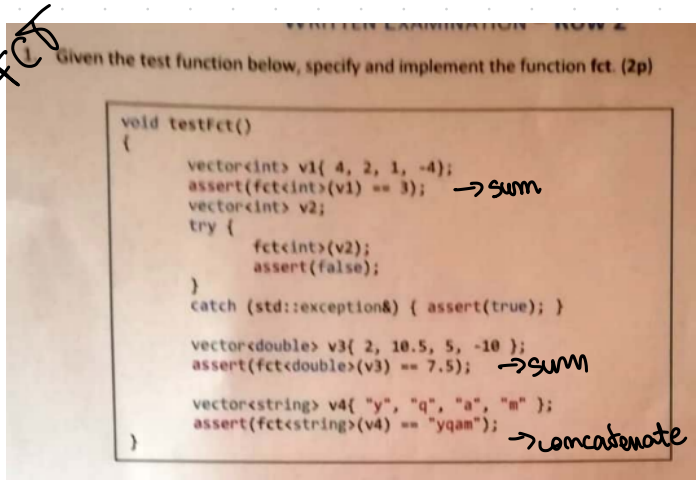
Paragraph → inh. Element
→ text
→ getHTMLString
→ constr.

Builder → template
→ op +=
→ op <<
→ op *

```
class HTMLElement
{ public: virtual std::string getHTMLString() const=0;
};
class HTMLParagraph: public HTMLElement
{ private: std::string text;
  public: HTMLParagraph(const std::string& text) : text(t) {}
          std::string getHTMLString() const override { return "<p>"+text+"</p>";}
};
class HTMLImage : public HTMLElement
{ private: std::string src;
  public: HTMLImage(const std::string s) : src(s) {}
          std::string getHTMLString() const override { return "<img>"+src+"</img>";}
};
template <typename T>
class HTMLBuilder
{ private: std::vector<T> elements;
  public: HTMLBuilder& operator+=(T elem)
          { if(elem==nullptr) throw std::runtime_error("Cannot add a null element!");
            elements.push_back(elem);
            return *this;
          }
          friend std::ostream& operator<<(std::ostream& out, const
                HTMLBuilder<T>& builder)
          { out<<"<html><body>";
            for(auto& e:builder.elements)
              out<<e->getHTMLString();
            out<<"</body></html>";
            return out;
          }
};
```

---

fct



Given the test function below, specify and implement the function **fct**. (2p)

```
void testFct()
{
    vector<int> v1{ 4, 2, 1, -4};
    assert(fct<int>(v1) == 3);   → sum
    vector<int> v2;
    try {
        fct<int>(v2);
        assert(false);
    }
    catch (std::exception&) { assert(true); }

    vector<double> v3{ 2, 10.5, 5, -10 };
    assert(fct<double>(v3) == 7.5);   → sum

    vector<string> v4{ "y", "q", "a", "m" };
    assert(fct<string>(v4) == "yqam");   → concatenate
}
```

fct → template
→ empty → exception
→ int, double, string

```
template <typename T>
T fct(std::vector<T>& v)
{ if(v.empty()) throw std::exception();
  T result{};
  for(auto& x:v)
    result = result+v;
  return result;
}
```

SMARTPOINTER VECTOR

Define the classes **SmartPointer** and **Vector** such that the following C++ code is correct. Enumerate the data members (including types) and the prototypes of all methods and implement the method allowing the operation emphasized in the code below. **(2p)**

```
void function1() {
      SmartPointer<int> i1{ new int{ 1 } };
      SmartPointer<int> i2{ new int{ 2 } };
      SmartPointer<int> i3{ new int{ 3 } };
      Vector<SmartPointer<int>> v1{};
      v1.add(i1).add(i2).add(i3);
      for (auto e : v1)
            cout << *e << ", "; // prints 1, 2, 3

      SmartPointer<string> s1{ new string{ "A" } };
      SmartPointer<string> s2 = s1;
      SmartPointer<string> s3{ new string{ "C" } };
      Vector<SmartPointer<string>> v2{};
      v2.add(s2).add(s1);
      try {
            v2 = v2 - s2;  // IMPLEMENT THIS OPERATION
            v2 = v2 - s3;  // IMPLEMENT THIS OPERATION
      }
      catch (std::runtime_error& ex) {
            cout << ex.what(); // prints: "Element does not exist!"
      }
} // memory is correctly deallocated
```

SmartPointer → template
→ constr
→ copy constr
→ destr
→ assign. op =
→ op *
→ op == (for later)

Vector → template
→ add()
→ begin/end
→ op -
→ constr

```
template <typename T>
class SmartPointer
{ private: T* elem;
  public : SmartPointer(T* elem = nullptr) : elem(elem) {}
           SmartPointer(const SmartPointer<T>& other) :
              elem (new T(*other.elem)) {}
           SmartPointer& operator=(const SmartPointer& other)
           { if (this != &other)
             { delete elem;
               elem = new T(*other.elem);
             }
             return *this;
           }
           ~SmartPointer() { delete elem; }
           T& operator*() const { return *elem; }
           bool operator==(const SmartPointer& other) const
           { return *elem == *other.elem; }
};
template <typename T>
class Vector
{ private: std::vector<T> data;
  public : Vector& add(const T& e)
           { data.push_back(e);
             return *this;
           }
           Vector operator-(const T& e) const
           { Vector result; bool found = false;
             for(auto& x: data)
               if(!(x==e)) result.push_back(x);
               else found = true;
             if(!found) throw std::runtime_error
                          ("Element does not exist");
             return result;
           }
           auto begin() { return data.begin(); }
           auto end() { return data.end(); }
};
```

SET SMARTPOINTER

Define the classes **SmartPointer** and **Set** such that the following C++ code is correct. Enumerate the data members (including types) and the prototypes of all methods and implement the method allowing the operation shown emphasized in the code below. **(1.4p)**

```
void function2() {
      SmartPointer<string> s1{ new string{ "A" } };
      SmartPointer<string> s2 = s1;
      SmartPointer<string> s3{ new string{ "C" } };
      Set<SmartPointer<string>> set1{};

      try {
            set1 = set1 + s1;  // IMPLEMENT THIS OPERATION
            set1 = set1 + s2;  // IMPLEMENT THIS OPERATION
      }
      catch (std::runtime_error& ex) {
            cout << ex.what(); // prints: "Element already exists!"
      }

      SmartPointer<int> i1{ new int{ 1 } };
      SmartPointer<int> i2{ new int{ 2 } };
      SmartPointer<int> i3{ new int{ 3 } };
      Set<SmartPointer<int>> set2{};
      set2 = set2 + i1;  // IMPLEMENT THIS OPERATION
      set2 = set2 + i2;  // IMPLEMENT THIS OPERATION
      set2 = set2 + i3;  // IMPLEMENT THIS OPERATION
      set2.remove(i1).remove(i3);
      for (auto e : set2)
            cout << *e << ", "; // prints 2,
} // memory is correctly deallocated
```

*already given, so I'm not going to implement it

SmartPointer → template
→ constr.
→ destr.
→ copy constr.
→ assign. op=
→ op== (used later)
→ op*

Set → template
→ op+
→ begin/end
→ remove

PERSON
MULTIDICTIONARY

WRITTEN EXAMINATION – ROW 2

1. Define the classes **Person** and **MultiDictionary** such that the following C++ code is correct. Enumerate the data members (including types), the prototypes of all methods (and other functions) and implement the method allowing the operation emphasized in the code below. (1.4p)

```
void function()
{
        MultiDictionary<int, string> d1{};
        d1.add(1, "a").add(2, "b").add(1, "c").add(3, "d");
        d1.print(std::cout); // prints 1 a; 1 c; 2 b; 3 d
        try     {
                d1.erase(4, "w"); assert(false);   }
        catch (std::runtime_error& e) {
                try {
                        assert(strcmp(e.what(), "Key does not exist!") == 0);
                        d1.erase(2, "w"); assert(false);   }
                catch (std::runtime_error& e) {
                        assert(strcmp(e.what(), "Given value does not exist for given key!") == 0);
                }
        }
        d1.erase(1, "a"); d1.print(std::cout); // prints 1 c; 2 b; 3 d
        MultiDictionary<string, Person> d2{};
        Person p1{ 23, "Mircea"}; Person p2{ 20, "Ioana" };
        std::cout << p2; // prints: Ioana is 20 years old
        d2.add("a", p1).add("a", p2);
        d2.print(std::cout); // prints: a Mircea is 23 years old; a Ioana is 20 years old
        assert(d2.sizeForKey("a") == 2);  assert(d2.sizeForKey("b") == 0);
}
```

...the result of the execution of the following C++ programs. Justify your answers. (4 x0.9p)

Person → int age, string name
→ op <<
→ constr.

MultiDictionary → template <T1, T2>
→ add()
→ print(ostream)
→ erase()
→ sizeForKey()

```
class Person
{ private: int age;
           std::string name;
  public: Person(int a, std::string n) : age(a), name(n) {}
          friend std::ostream& operator<<(std::ostream& cout
                                          Person& p)
          { cout<<p.name<<" is "<<p.age<<" years old\n";
            return cout; }
};
template <typename key, typename value>
class MultiDictionary
{ private: std::vector<std::pair<key, value>> data;
  public: MultiDictionary& add(key& k, value& v)
          {   data.push_back(k,v);
              return *this;
          }
        void erase(const key& k, const values& v)
          { bool foundkey = false, foundPair = false;
            std::vector<std::pair<key, value>> newData;
            for(const auto& [key, value] : data)
              { if(key==k)
                  { foundkey = true;
                    if(value==v)
                      { foundPair=true;
                        continue;
                      }
                  }
                newData.push_back(key, value);
              }
            if(!foundkey) throw std::runtime_error(key
                                                   does not exist!);
            if(!foundPair) throw std::runtime_error(Given
              value does not exist for given key);
            data = newData;
          }
        void print(std::ostream& out) const
          { for(auto& [key, value] : data)
              out << key<<" "<<value<<'
                                      !: ";
          }
        int sizeForKey(const key& k) const
          { int count=0;
            for(auto&[key,_] :data)
                if(key==k) count++;
            return count;
          }
};
```

Exercise 2

2. Determine the result of the execution of the following C++ programs. **Justify your answers. (4 x 0.9p)**

```
// a)
int except(int v) {
        if (v < 0) { throw string{ "1 " }; }
        else if (v > 0) {
                throw std::runtime_error{"2 "};
        }
        return 0;
}
int main()
{
        try {
                cout << except(10 < 2) << " "; -> 0␣   (0 = false)
                cout << except(-2) << " ";     -> 1␣ -> stop
                cout << except(2) << " ";
        }
        catch (std:: runtime_error& e) {
                cout << e.what();
        }
        catch (string x) {
                cout << x;
        }
        cout << "Done ";  -> Done
        return 0;
}
```

Result: 0␣ 1␣ Done

```
// b)
int main()
{
        vector<int> v{ 10, 9, 8, 7, 6, 5 };   (4 1)
        vector<int>::iterator it = v.begin() + 2;
        v.erase(it);
        it = v.begin() + 3;
        *it = 11;
        it++;
        *it = 1;
        deque<int> x;                          (9 7 11 1)
        std::copy_if(v.begin(), v.end(),
front_inserter(x), [](int a) { return a % 2 ==
1; });
        for (auto a : x)
                cout << a << " ";

        return 0;
}
```

-> insertion order is reversed

Result: 1␣ 11␣ 7␣ 9␣

already given

```
// c)
class B {
public:
        B() {}
        B(const B&) { cout << "copy "; }
        virtual B f() { cout << "B.f "; return *this; }
        virtual ~B() { cout << "~B "; }
};

class D : public B
{
private:
        B* b;
public:
        D(B* _b) : b{ _b } { cout << "D "; }
        B f() override { cout << "D.f "; return b->f();
}
};

int main()
{
        B* b = new B();
        B* d = new D{ b };   -> Δ
        d->f();   -> Δ.f  B.f  copy (calls copy constr.)
        delete d; -> ~B
        delete b; -> ~B
        return 0;
}
```

Result: Δ Δ.f B.f copy ~B ~B

already given

```
// d)
class A {
private:
        int* x;
public:
        static int noOfInstances;

        A(int _x = 5) {
                x = new int{ _x };
                noOfInstances++;
        }
        int get() { return *x; }
        void set(int _x) { *x = _x; }
        ~A() { delete x; }
};

int A::noOfInstances = 0;

int main()
{
        A a1, a2;   -> a1.x = 5, a2.x = 5
        cout << a1.noOfInstances << " "; -> 2
        A a3 = a1; -> no copy constructor => shallow copy
        cout << A::noOfInstances << " "; -> 2   (a3.x = a1.x)
        a1.set(10);
        cout << a1.get() << " "; -> 10   (a1.x = 10 = a3.x)
        cout << a2.get() << " "; -> 5
        cout << a3.get() << " "; -> 10
                                  ↳ same memory with a1
        return 0;
}
```

Result: 2 2 10 5 10 + error

2. Determine the result of the execution of the following C++ programs. Justify your answers. (4x0.9p)

```
// a)
class Base {
private:
        virtual void f() { std::cout << "Base" <<
"\n"; }
public:
        virtual ~Base() { f(); }
        void g() { f(); }
};

class Derived : public Base {
private:
        void f() { std::cout << "Derived" <<
"\n"; }
public:
        void h() { Base::g(); cout << "Function
h" << "\n"; }
        ~Derived() { f(); }
};

int main() {
        Base* b = new Derived{};
        b->g();
        Derived *d = dynamic_cast<Derived*>(b);
        if (d != nullptr)
                d->h();
        delete b;
        return 0;
}
```

→ Derived
Derived Function h
→ Derived Base

```
// b)
class Exception1 {
public:
        Exception1() { cout << "Exception1 "; }
        virtual void fct() { cout << "Ex1 "; }
        virtual ~Exception1() { cout << "~Exception1
"; }
};

class Exception2 : public Exception1 {
public:
        void fct() override { cout << "Ex2 "; }
        virtual ~Exception2() { cout << "~Exception2
"; }
};

string f(int x) {
        cout << "Hi ";
        if (x % 2 == 0)
                throw Exception1{};
        else
                throw Exception2{};
        return string{ "Bye " };
}

int main() {
        try {
                cout << f(5);
                cout << f(2);
        }
        catch (Exception1& ex) { ex.fct(); }
        catch (Exception2& ex2) { ex2.fct(); }
        return 0;
}
```

Hi Exception 1 — no exec
↳ Ex 2
~Exception 2
~Exception 1

Result: Derived Derived Function h Derived Base

Result: Hi Exception 1 Ex 2 ~Exception 2 ~Exception 1

```
// c)
class A {
private:
        int x;
public:
        A(int _x = 0) : x{ _x } {}
        int get() const { return x; }
        friend class B;
};

class B {
private:
        A a;
public:
        B(const A& _a) : a{ _a } {}
        B& operator+(const A& _a) {
                a.x += _a.x;
                return *this;
        }
        int get() { return a.x; }
};

int main() {
        A a1{ 1 }, a2{ 2 };
        cout << a1.get() << " " << a2.get() << "
";
        B b1{ a1 };
        B b2 = b1 + a2 + a1;
        cout << b1.get() << " " << b2.get();
        return 0;
}
```

1 2
$b_1.x = 1$
$b_1.x = b_1.x + a_2.x = 1+2 = 3$
$b_1.x = b_1.x + a_1.x = 3+1 = 4$
$b_2.x = b_1.x = 4$
4
4

```
// d)
int main() {
        vector<string> v{ "rain", "in", "spain",
"falls", "mainly", "plain" };
        sort(v.begin(), v.end(), [](string a, string
b) { return a > b; });
        vector<string>::iterator it = v.begin() + 4;
        *it = "brain";
        it = v.begin();
        while (it != v.end())
        {
                string aux = *it;
                char c1 = aux[aux.size() - 3];
                char c2 = aux[aux.size() - 2];
                char c3 = aux[aux.size() - 1];
                if (c1 == 'a' && c2 == 'i' && c3 ==
'n')
                        cout << *it << "\n";
                it++;
        }
        return 0;
}
```

→ sorted desc
↳ spain rain plain mainly falls in
brain

→ spain
rain
plain
brain

Result: 1 2 4 4

done before

2. Determine the result of the execution of the following C++ programs. **Justify your answers. (2 x 1.5p)**

```
// a)
class A {
private:
	int* x;
public:
	static int noOfInstances;

	A(int _x = 0) {
		x = new int{ _x };
		noOfInstances++;
	}
	int get() { return *x; }
	void set(int _x) { *x = _x; }
	~A() { delete x; }
};

int A::noOfInstances = 0;

int main()
{
	A a1, a2;
	cout << a1.noOfInstances << " ";   2
	A a3 = a1; -shallow copy
	cout << A::noOfInstances << " ";   2
	a1.set(8);
	cout << a1.get() << " ";           8
	cout << a2.get() << " ";           0
	cout << a3.get() << " ";           8

	return 0;
}
```

Result: 2 2 8 5 8 + error

```
// b)
class B {
public:
	B() {}
	B(const B&) { cout << "copy "; }
	virtual B f() { cout << "B.f "; return
*this; }
	virtual ~B() { cout << "~B "; }
};

class D : public B
{
private:
	B* b;
public:
	D(B* _b) : b{ _b } { cout << "D "; }
	B f() override { cout << "D.f "; return
b->f(); }
};

int main()
{
	B* b = new B();
	B* d = new D{ b };   → D
	d->f();              → D.f   B.f copy
	delete d;            ~B
	delete b;            ~B
	return 0;
}
```

Result: D D.f B.f copy ~B ~B

2. Determine the result of the execution of the following C++ programs. If there are any errors, indicate the exact place where the errors occur. **Justify your answers. (4 x 0.75p)**

```
// a)
int main()
{
	vector<int> v{ 1, 2, 3, 4, 5 };
	vector<int>::iterator it =
std::find(v.begin(), v.end(), 4);
	v.insert(it, 8);        1 2 ~~3~~ 8 4 5
	it = v.begin() + 2;          10
	*it = 10;
	vector<int> x;
	std::copy_if(v.begin(), v.end(),
back_inserter(x), [](int a) { return a % 2 ==
0; });
	for (auto a : x)
		cout << a << " ";

	return 0;                2 10 8 4
}
```

```
// b)
class Ex1 {
public:
	Ex1() { cout << "Exception1 "; }
	Ex1(const Ex1& ex) { cout << "Copy_ex1 ";
}
};

class Ex2 : public Ex1 {
public:
	Ex2() { cout << "Exception2 "; }
	Ex2(const Ex2& ex) { cout << "Copy_exc2
"; }
};
void except(int x) {
	if (x < 0)
		throw Ex1{};
	else if (x == 0)
		throw Ex2{};
	cout << "Done ";
}
int main()
{
	try {
		cout << "Start ";          → Start
		try {
			except(0);            → Exception1
		                          → Exception2
		}
		catch (Ex1& e) {}         → copy_exc2
		except(-2);               → Exception1
	} catch (Ex1 e) {}            → copy_exc1
	return 0;
}
```

```cpp
class B
{
public:
        void f() { cout << "B.f "; }
        virtual ~B() {}
};

class D1 : public B
{
public:
        virtual void f() { cout << "D1.f "; }
        virtual ~D1() {}
};

class D2 : public D1
{
public:
        void f() { cout << "D2.f "; }
};

int main()
{
        B* b1 = new B{};  b1->f(); delete b1;
        B* b2 = new D1{}; b2->f(); delete b2;
        B* b3 = new D2{}; b3->f(); delete b3;
        D1* d = new D2{}; d->f();  delete d;
        return 0;
}
```

B.f B.f B.f D2.f

because f() is not virtual in B

```cpp
class Vector {
        int* elems;
        int size;

public:
        Vector() : size{ 0 } { elems = new int[10]; }
        void add(int elem) {
                elems[size++] = elem; }
        int& operator[](int pos) {
                if (pos < 0 || pos >= size)
                        throw std::runtime_error{ "Index out of bounds." };
                return elems[pos];
        }
        ~Vector() { delete [] elems; }
};

int main()
{
        Vector v1;
        v1.add(0);
        v1.add(1);
        Vector v2 = v1;
        try {
                v1[0] = 2;
                cout << v1[0] << " " << v1[1] << " ";
                cout << v2[0] << " " << v2[1] << " "; }
        catch (std::runtime_error& e) { cout << e.what(); }
        return 0;
}
```

$v_1 = [0, 1]$

shallow copy

2 1 2 1

error / crash

```cpp
// c)
int except(int v) {
        if (v < 0) { throw 1; }
        else if (v > 0) {
                throw std:: runtime_error{"2"};
        }
        return 0;
}

int main()
{
        try {
                cout << except(5 < 5) << " ";
                cout << except(-2) << " ";
                cout << except(2) << " ";
        }
        catch (std:: runtime_error& e) {
                cout << "2 ";
        }
        catch (int x) {
                cout << "- ";
        }
        cout << "+ ";
        return 0;
}
```

false => 0 → 0 1 // 0

→ no exec

→ no exec

// - → -

// + → +

```cpp
// d)
int main()
{
        vector<int> v{ 0, 2, 4, 6, 8, 10 };
        vector<int>::iterator it = v.begin() + 2;
        v.erase(it);
        it = v.begin() + 2;
        *it = 7;
        it++;
        *it = 0;
        deque<int> x;
        std::copy_if(v.begin(), v.end(), front_inserter(x), [](int a) { return a % 2 == 0; });
        for (auto a : x)
                cout << a << " ";

        return 0;
}
```

7 0

10 0 2 0

Result: 0 - +

2. Determine the result of the execution of the following C++ programs. If there are any errors, indicate the exact place where the errors occur. **Justify your answers. (4 x 0.75p)**

```
// a)
int fct(string s)
{
        if (s == "") { throw string{ "2" }; }
        cout << "1 ";
        if (s.size() > 4) { throw
std::runtime_error("Not empty"); }
        return 0;
}

int main()
{
        try {
                cout << fct("Hi") << " ";
                cout << fct("Hello") << " ";
                cout << fct("") << " ";
        }
        catch (string& e) {
                cout << e;
        }
        catch (std::runtime_error& e) {
                cout << e.what();
        }
        return 0;
}
```

→ 1 0
→ 1 Not Empty

Result: 101 Not Empty

```
// b)
int main()
{
        vector<int> v{ 1, 2, 3, 4, 5 };
        vector<int>::iterator it =
std::find(v.begin(), v.end(), 4);
        v.insert(it, 7);
        it = v.begin() + 2;
        *it = 11;
        vector<int> x;
        std::copy_if(v.begin(), v.end(),
back_inserter(x), [](int a) { return a % 2 ==
1; });
        for (auto a : x)
                cout << a << " ";
        return 0;
}
```

1 11 7 5

```
// c)
class B {
public:
        B() {}
        B(const B& b) { cout << "copy "; }
        virtual void f() { cout << "B.f "; }
        void g(B b) { cout << "B.g "; };
        virtual ~B() { cout << "~B "; }
};

class D: public B {
public:
        D(){}
        void f() { B::f(); cout << "D.f "; }
        void g(D d) { B::g(d); cout << "D.g "; }
};

int main()
{
        B* b = new B{};
        B* d = new D{};
        d->f();
        d->g(*b);
        delete b;
        delete d;

        return 0;
}
```

B.f D.f
copy B.g
~B
~B
~B

```
// d)
class A {
private:
        int* x;
public:
        A(int _x = 0) {
                x = new int{ _x };
        }

        A(const A& a) {
                x = new int{ *a.x };
        }

        int get() { return *x; }
        void set(int _x) { *x = _x; }

        ~A() { delete x; }
};

int main()
{
        A a1, a2; a1.set(8);
        A a3; a3 = a1;
        A a4 = a1;
        a1.set(5);
        cout << a1.get() << " ";
        cout << a2.get() << " ";
        cout << a3.get() << " ";
        cout << a4.get() << " ";

        system("pause");
        return 0;
}
```

a1.x=0 → a2.x=0
→ a1.x = 8
→ a3.x = a1.x = 8
→ a4.x = 8
→ a1.x = 5 = a3.x
5
0
5
8

Shallow copy ⇒ error / crash

```
// a)

#include <iostream>
using namespace std;

class B {
public:
    virtual void f() { cout << "B.f "; }
    virtual ~B() { cout << "~B "; }
};
class D : public B {
private:
    B& b;
public:
    D(B& _b) : b(_b) { cout << "D "; b.f(); }
    void f() override { cout << "D.f "; }
    ~D() { cout << "~D "; }
};

int main()
{
    B* b1 = new B{};
    b1->f();
    B* b2 = new D{ *b1 };
    b2->f();
    delete b2;
    delete b1;
    return 0;
}
```

B f

D B.f

D f

~D ~B

~B

```
// b)

#include <iostream>
#include <string>
using namespace std;

string except(int x)
{
        if (x < 0)
                throw string{ "Negative " };
        return "Positive ";
}

int main()
{
    cout << "One ";
    try {
            cout << except(3);
            cout << except(-2);
            cout << except(5);
    }
    catch (string& ex) { cout << ex << " "; }
    return 0;
}
```

→ One

Positive

negative

```
// c)
#include <iostream>
#include <string>
```

```
// d)
#include <iostream>
#include <string>
```

```
using namespace std;
template <typename T, typename U>
U fct2(T a, T b, U x, U y)
{
        cout << a << " ";
        cout << b << " ";
        if (a == b)
                return x + y;
        return x;
}

class A {
        int a;
public:
        A(int _a) : a{ _a } {}
};

int main()
{
    cout << fct2<int, int>(10, 10, 5, 5) << " ";
    cout << fct2<double, int>(10, 10.5, 5, 5) <<
" ";
    cout << fct2<int, string>(-2, -2, "Good ",
"luck!");
    cout << fct2<int, A>(1, 1, A{ 2 }, A{ 3 });
    return 0;
}
```

10 10 10

10 10.5 5

-2 -2 Good luck!

1 1 error since ==, + are not declared for objects of type A

```
#include <vector>
using namespace std;
int main()
{
    vector<string> str{ "a", "b", "", "" };
    str.erase(str.begin() + 2);
    vector<string>::iterator it =
str.begin();
    str.insert(it + 2, "b");
    str.insert(str.end() - 1, "a");
    str.pop_back();
    it = str.begin();
    while (it != str.end())
    {
        cout << *it << " ";
        it++;
    }
    return 0;
}
```

b a

a b b a

```cpp
#include <iostream>
#include <string>
using namespace std;

class Exception1 {
public:
    virtual void fct() { cout << "Ex1.fct "; }
    virtual ~Exception1() { cout << "~Ex1 "; }
};

class Exception2 : public Exception1 {
public:
    void fct() override { cout << "Ex2.fct "; }
    virtual ~Exception2() { cout << "~Ex2 "; }
};

int f(string x) {
    if (x[0] < 'a') throw Exception1{};
    else throw Exception2{};
    return -1;
}

int main() {
    try {
        cout << f("exam");
        cout << f("test");
    }
    catch (Exception1& ex) {
        ex.fct();
    }
    catch (Exception2& ex2) {
        ex2.fct();
    }
    return 0;
}
```

→ Ex(2)

→ Ex1.fct
~Ex2
~Ex1

Result : Ex2.fct
~Ex2
~Ex1

```cpp
#include <iostream>
using namespace std;

class A {
    int x;
public:
    A(int _x = 0) : x(_x) {}
    int get() const { return x; }
    friend class C;
};

class C {
private:
    A a;

public:
    C(const A& _a) : a(_a) {}

    C& operator*(const A& _a) {
        a.x *= _a.x;
        return *this;
    }

    int get() { return a.x; }
};

int main() {
    A a1, a2(2);
    cout << a1.get() << " " << a2.get() << " ";  // 0 2
    C c1(a2);       // a2 = 2
    C c2 = c1 * a1; // a1 = 0 → c2 = 2 * 0 = 0
    C c3 = c1 * a2; // a2 = 2 → c1 updated to 2 * 2 = 4 → c3 = 4
    cout << c1.get() << " " << c2.get() << " " << c3.get();

    return 0;
}
```

0 2

a2=2 a1=0 ⇒ c2=0

a2=2 ⇒ c1=0 ⇒ c3=0

0 0 0

Result : 0 2 0 0 0

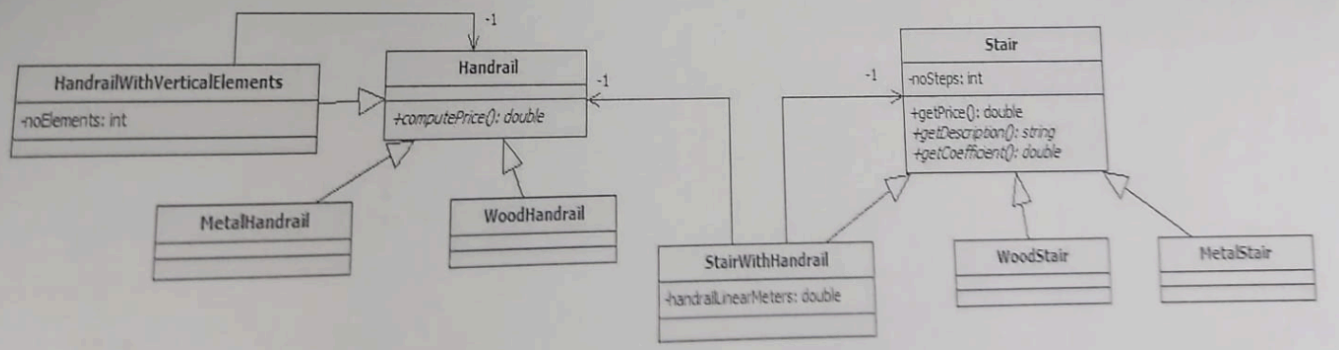
Exercise 3

* Section - Checker → on guide
* Event - GUI - already given
* Icecream - previously done (above ↑)

To do: menu ✓
encoder ✓
handrail ✓
channel-fax ✓
main ✓
sale ✓

HANDRAIL

3. Write a C++ application for building stairs **(4p)**, as follows:
   a. The class **Handrail** is abstract, it contains the abstract method *computePrice*. **(0.2p)**
   b. The classes **WoodHandrail** and **MetalHandrail** represent specific types of handrails. The price for one linear meter of wood handrail is 2, while for metal handrails this price is 2.5. **(0.3p)**
   c. The price for one linear meter of handrail with vertical elements (**HandrailWithVerticalElements**) is the price of the aggregated handrail, to which the price of elements is added. The price for one element it 5. **(0.5p)**
   d. The class **Stair** represents a generic stair. The price of a simple stair is computed as the coefficient multiplied by the stair's number of steps. **(0.5p)**
   e. The description for a wood stair (**WoodStair**) is "wood stair", while for a metal one (**MetalStair**), this description is "metal stair". The coefficient for a wood stair is 1.5, while for a metal one it is 2. **(0.6p)**
   f. The price of a stair with handrail (**StairWithHandrail**) is the price of the aggregated stair, to which the price of the aggregated handrail multiplied by the number of linear meters is added. The coefficient for a stair with handrail is 1, while the description is the description of the aggregated stair, to which the number of linear meters of handrail is added. **(0.7p)**
   g. Create the following objects: a simple wood stair, with 20 steps; a metal stair, with 10 steps, with 5 linear meters of metal handrail; a wood stair with 10 steps, with 5 linear meters of wood handrail, with 10 vertical elements. Print the description and price for each stair. Make sure the memory is correctly managed. **(1.2p)**

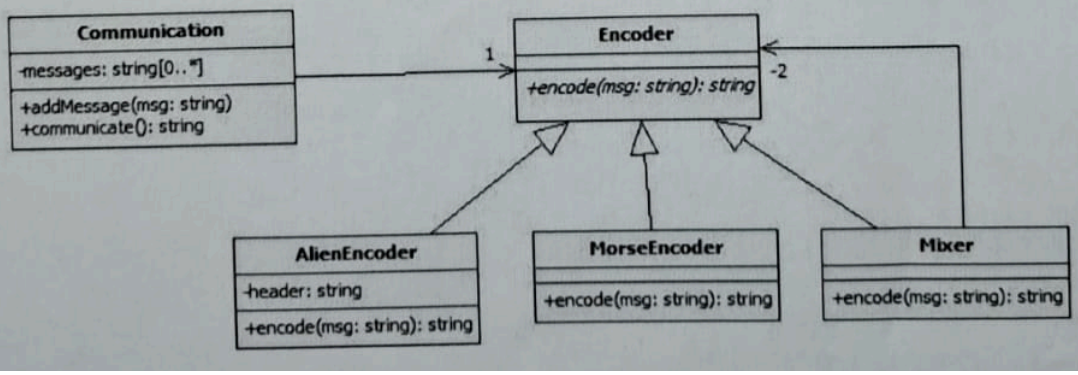



```
class Handrail
{ public: virtual double computePrice() = 0;
          virtual ~Handrail() = default;
};
class WoodHandrail : public Handrail
{ public: double computePrice() override
          { return 2.0;
          }
};
class MetalHandrail : public Handrail
{ public: double computePrice() override
          { return 2.5;
          }
};
class HandrailWithVerticalElements: public Handrail
{ private: Handrail * base;
           int noElements;
  public: HandrailWithVerticalElements(Handrail *b, int n) : base(b), noElements(n){}
          ~HandrailWithVerticalElements() { delete base; }
```

```
        double computePrice() override
        { return base->computePrice() + 5.0 * noElements; }

class Stair
{ private: int noSteps;
  public: ~Stair() = default;
          Stair(int n): noSteps(n) {}
          virtual string getDescription() = 0;
          virtual double getCoefficient() = 0;
          double getPrice()
          { return noSteps * getCoefficient(); }
};

class WoodStair: public Stair
{ public: WoodStair(int steps): Stair(steps) {}
          double getCoefficient() override { return 1.5; }
          string getDescription() override { return "wood stair"; }
};

class MetalStair: public Stair
{ public: MetalStair(int steps): Stair(steps) {}
          double getCoefficient() override { return 2.0; }
          string getDescription() override { return "metal stair"; }
};

class StairWithHandrail: public Stair
{ private: Stair* base;
          Handrail* handrail;
          double handrailLinearMeters;
  public: StairWithHandrail(Stair* b, Handrail* h, double m): Stair(b->noSteps), base(b), handrail(h), handrailLinearMeters(m) {}
          double getCoefficient() override { return 1.0; }
          string getDescription() override { return base->getDescription() + "with handrail" + to_string
                                                   (handrailLinearMeters) + "m"; }
          double getPrice override()
          { return base->getPrice() + handrail->computePrice() * handrailLinearMeters; }
         ~StairWithHandrail() { delete base; delete handrail; }
};

int main()
{ Stair* s1 = new WoodStair(20);
  Stair* s2 = new (StairWithHandrail (new MetalStair(10), new MetalHandrail(), 5);
  Stair* s3 = new(StairWithHandrail( new WoodStair(10), new HandrailWithVerticalElem(new WoodHand(),10)
  vector<Stair*> stairs = {s1, s2, s3};
  for(Stair* s: stairs)
  { cout << s->getDescription() << " costs" << s->getPrice() << "\n";
    delete s;
  }
  return 0;
}
```

ENCODER

3. Write a C++ application for message encoding, as follows:
   a. The class **Encoder** is abstract, its role is to encode a message in a specific way. **(0.3p)**
   b. Create the class **AlienEncoder**. The method *encode* in this class adds the given header to the beginning of the received message. **(0.4p)**
   c. Create the class **MorseEncoder**. The method *encode* in this class replaces each character in the message with the character ".". **(0.3p)**
   d. Create the class **Mixer**. The method *encode* in this class creates a message by alternating the letters in the two aggregate encoders. **(1p)**
   e. The class **Communication** can contain one or more messages. The method ***communicate*** returns all the given messages, encoded. Before encoding, the messages are sorted alphabetically. **(0.8p)**
   f. Create 2 Communication objects: one uses a MorseEncoder and the other one uses a Mixer. The Mixer mixes an AlienEncoder (with "ET" as header) with a MorseEncoder. Each Communication object contains 2 messages. Show the encoded messages of both objects. Take memory management into consideration and implement it correctly **(1.2p)**

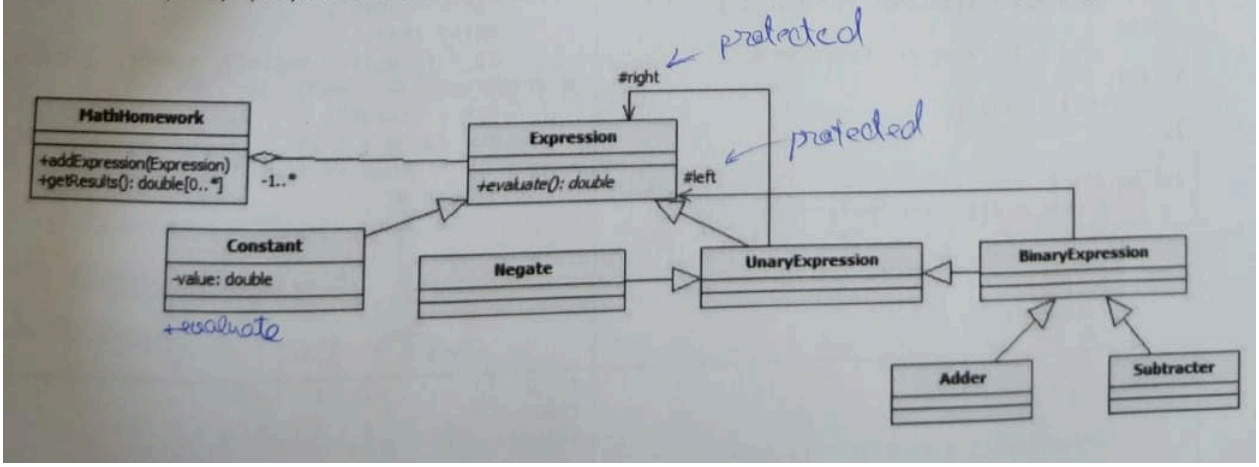



```
class Encoder
{ public: virtual std::string encode(string msg);
          virtual ~Encoder() = default;
};
class AlienEncoder : public Encoder
{ private: string header;
  public : AlienEncoder(string &h) header(h) {}
           string encode(string msg) override { return header+msg;}
};
class MorseEncoder: public Encoder
{ public : string encode(string msg) override { return string(msg.size(), '.'); }
};
class Mixer : public Encoder
{ private : Encoder * e1 ;
            Encoder * e2 ;
  public : Mixer(encoder *e1, encoder * e2) : e1(e1), e2(e2){}
           string encode (string msg) override
              { string a = e1->encode(msg);
                string b = e2->encode(msg);
                string result;
                for(int i=0; i < max(a.size() , b.size()); ++i)
                    { if (i< a.size()) result += a[i];
                      if(i < b.size()) result += b[i];
                    }
                    return result;
              }
           ~Mixer()
             { delete e1; delete e2;}
        };
```

```
class Communication
{ private: vector<string> message;
           Encoder * encoder;
  public: Communication(Encoder *e): encoder(e) {}
          void addMessage(string &msg) {messages.push_back(msg);}
          string communicate() {
              sort(messages.begin(), messages.end());
              string result;
              for(auto * m: messages)
                  result += encoder->encode(m) + "\n";
              return result;
          }
          ~Communication() {delete encoder;}
};
int main()
{ Communication *c1 = new Communication(new HorseEncoder());
  c1->addMessage("Mars");
  c1->addMessage("Earth");
  Encoder * mix = new Mixer(new AlienEncoder("ET"), new HorseEncoder());
  Communication *c2 = new Communication(mix);
  c2->addMessage("Mars");
  c2->addMessage("Earth");
  cout << c1->communicate();
  cout << c2->communicate();
  delete c1;
  delete c2;
  return 0;
}
```

MATH

3. Write a C++ application for computing mathematical expressions, as follows:
   a. The class **Expression** is abstract, it contains the pure virtual function *evaluate()*. **(0.2p)**
   b. The class **Constant** represents a constant and contains a value. The evaluation result of a constant expression is its own value. **(0.3p)**
   c. The classes **UnaryExpression** and **Negate** each aggregate an expression and the first returns the evaluation of the aggregated expression, while the second returns the negation the evaluation of the aggregated expression. **(0.7p)**
   d. The class **BinaryExpression** aggregates two other expressions. **(0.3p)**
   e. The classes **Adder** and **Subtracter** are evaluated as the sum and the difference, respectively, of their aggregated expressions. **(0.8p)**
   f. The class **MathHomework** allows the creation of a homework with several expressions that need to be evaluated. The method getResults() will return all the results of the contained expressions. **(0.5p)**
   g. Write a function which creates homework that must evaluate the following two expressions: *-5 + (9 - 3)* and *-(4 + 2) - (-10)*. Print the results of the evaluation. Make sure the memory is correctly managed. **(1.2p)**



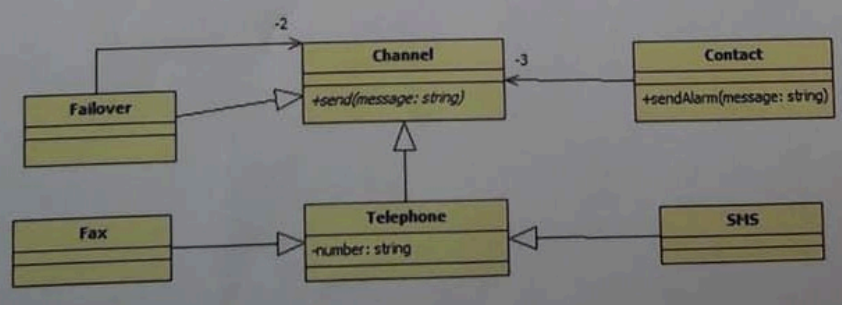
```
class Expression {
public:
    virtual double evaluate() const = 0;
    virtual ~Expression() = default;
};

class Constant : public Expression {
    double value;

public:
    Constant(double v) : value(v) {}

    double evaluate() const override {
        return value;
    }
};

class UnaryExpression : public Expression {
protected:
    Expression* expr;

public:
    UnaryExpression(Expression* e) : expr(e) {}

    ~UnaryExpression() {
        delete expr;
    }
};

class Negate : public UnaryExpression {
public:
    Negate(Expression* e) : UnaryExpression(e) {}

    double evaluate() const override {
        return -expr->evaluate();
    }
};
class BinaryExpression : public Expression {
protected:
    Expression* left;
    Expression* right;

public:
    BinaryExpression(Expression* l, Expression* r) :
left(l), right(r) {}

    ~BinaryExpression() {
        delete left;
        delete right;
    }
};
```

```
class Adder : public BinaryExpression {
public:
    Adder(Expression* l, Expression* r) :
BinaryExpression(l, r) {}

    double evaluate() const override {
        return left->evaluate() + right->evaluate();
    }
};

class Subtracter : public BinaryExpression {
public:
    Subtracter(Expression* l, Expression* r) :
BinaryExpression(l, r) {}

    double evaluate() const override {
        return left->evaluate() - right->evaluate();
    }
};

class MathHomework {
    vector<Expression*> expressions;

public:
    void addExpression(Expression* e) {
        expressions.push_back(e);
    }

    vector<double> getResults() const {
        vector<double> results;
        for (auto e : expressions)
            results.push_back(e->evaluate());
        return results;
    }

    ~MathHomework() {
        for (auto e : expressions)
            delete e;
    }
};
```

PAX CHANNEL

3. Write a C++ application for sending messages, as follows:
   a. The class **Channel** is abstract, it has a pure virtual function *send* and its role is to send the given message on the channel. **(0.3p)**
   b. The method *send* in class **Telephone** prints "dialing" and the telephone number. However, once in a while, according to a randomly generated number, this method will throw an exception, indicating that the line is busy. (To get a random number between 0 and 9, use: `srand(time(NULL)); int r = rand() % 10;`) **(0.5p)**
   c. Classes **Fax** and **SMS** try to access the telephone number and in case of success, the messages "sending fax" and "sending sms" are displayed, respectively. **(0.7p)**
   d. Class **Failover** tries to send the message using the two contained channels: if the first channel fails (the line is busy), it will try the second channel. **(0.75p)**
   e. The method *sendAlarm* in class **Contact** tries to send the message on the three contained channels, until the message is sent (a line that is not busy is found). **(0.75p)**
   f. Write a function which creates an object Contact, which contains the following three channels:
      - **Telephone**
      - **Fax**, if this is busy, then try **SMS**
      - **Telephone**, if this is busy, then try **Fax**, if this is busy, then try **SMS**.

   Make sure the memory is correctly managed. **(1p)**

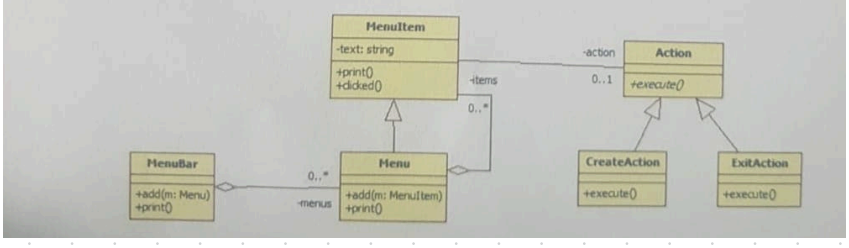


```
class Channel {
public:
    virtual void send(const string& msg) = 0;
    virtual ~Channel() = default;
};

class Telephone : public Channel {
    string number;

public:
    Telephone(const string& num) : number(num) {}

    void send(const string& msg) override {
        cout << "Dialing " << number << "\n";
        int r = rand() % 10;
        if (r < 4)  // simulate busy line (40%
chance)
            throw runtime_error("Line is busy!");
        cout << "Telephone: " << msg << "\n";
    }
};

class Fax : public Channel {
    Telephone* tel;

public:
    Fax(Telephone* t) : tel(t) {}

    void send(const string& msg) override {
        tel->send(msg);
        cout << "Sending fax: " << msg << "\n";
    }
};

class SMS : public Channel {
    Telephone* tel;

public:
    SMS(Telephone* t) : tel(t) {}

    void send(const string& msg) override {
        tel->send(msg);
        cout << "Sending sms: " << msg << "\n";
    }
};

class Failover : public Channel {
    Channel* first;
    Channel* second;

public:
    Failover(Channel* c1, Channel* c2) : first(c1),
second(c2) {}

    void send(const string& msg) override {
        try {
            first->send(msg);
        } catch (...) {
            second->send(msg);
        }
    }

    ~Failover() {
        delete first;
        delete second;
    }
};
```

```
class Contact {
    vector<Channel*> channels;

public:
    void add(Channel* c) {
        channels.push_back(c);
    }

    void sendAlarm(const string& msg) {
        for (Channel* c : channels) {
            try {
                c->send(msg);
                return;
            } catch (...) {
                continue;
            }
        }
        cout << "All channels failed!\n";
    }

    ~Contact() {
        for (auto c : channels)
            delete c;
    }
};

int main() {
    srand(time(nullptr));  // seed random

    Telephone* t1 = new Telephone("1234");
    Telephone* t2 = new Telephone("5678");

    Contact c;

    // First channel: Telephone t1
    c.add(new Telephone(*t1));

    // Second channel: Fax with t1, fallback to SMS
with t2
    c.add(new Failover(
        new Fax(t1),
        new SMS(t2)
    ));

    // Third channel: Telephone t2, fallback to Fax
t1, fallback to SMS t2
    c.add(new Failover(
        new Telephone(*t2),
        new Failover(
            new Fax(t1),
            new SMS(t2)
        )
    ));

    c.sendAlarm("Emergency!");

    delete t1;
    delete t2;

    return 0;
}
```

MENU

3. Write a C++ application which demonstrates the construction of menus, as follows:
   a. The class *Action* is abstract. The *execute* function will print "Create file" for a *CreateAction* and "Exit application" for an *ExitAction*. **(0.75p)**
   b. A *MenuItem* has a text and an associated action. When a menu item is displayed (function *print()*), its text is displayed. When a *MenuItem* is clicked (function *clicked()*), its text is displayed and its associated action (if this is valid), is executed. **(1p)**
   c. A *Menu* can contain several menu items. However, a menu can also be a menu item for another menu (a submenu). When a menu is displayed, besides showing the menu's text, all its submenus are displayed. **(0.75p)**
   d. A *MenuBar* contains several *Menus*. When a menu bar is displayed, all its elements are displayed. **(0.5p)**
   e. The main application will show a menu bar containing the menus *File* and *About*. None of these have any actions associated. *File* has *New* and *Exit* as submenus and New has *Text* and *C++* as submenus. Exit has an *ExitAction* associated, while *Text* and *C++* each have a *CreateAction*. Simulate the clicking of the following sequence: File -> New -> C++ and then Exit and show the actions that are executed. Take memory management into consideration and implement it correctly. **(1p)**

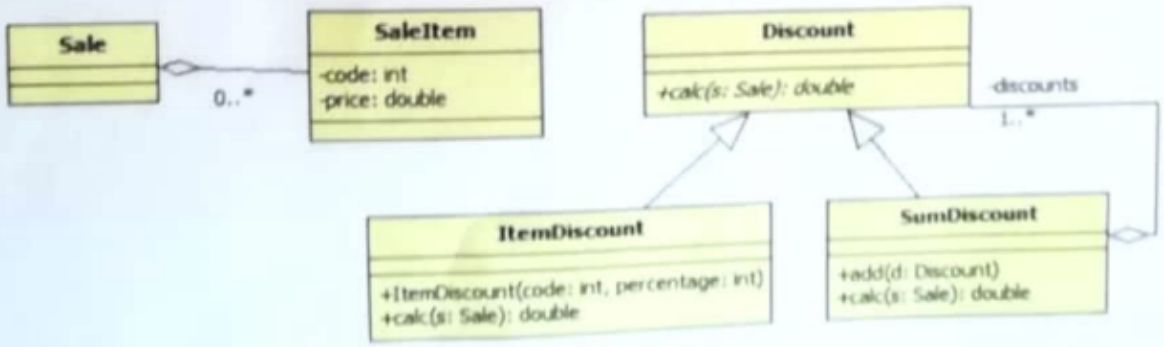


```
class Action {
public:
    virtual void execute() = 0;
    virtual ~Action() = default;
};

class CreateAction : public Action {
public:
    void execute() override {
        cout << "Create file\n";
    }
};

class ExitAction : public Action {
public:
    void execute() override {
        cout << "Exit application\n";
    }
};

class MenuItem {
    string text;
    Action* action;

public:
    MenuItem(const string& t, Action* a = nullptr)
        : text(t), action(a) {}

    void print() {
        cout << "- " << text << "\n";
    }

    void clicked() {
        cout << "Clicked: " << text << "\n";
        if (action) action->execute();
    }

    ~MenuItem() {
        delete action;
    }

    string getText() const { return text; }
};

class Menu : public MenuItem {
    vector<MenuItem*> items;

public:
    Menu(const string& name, Action* a = nullptr)
        : MenuItem(name, a) {}

    void add(MenuItem* item) {
        items.push_back(item);
    }

    void print() {
        MenuItem::print();
        for (auto item : items)
            item->print();
    }

    void clicked() {
        MenuItem::clicked();
        for (auto item : items)
            item->print();
    }

    MenuItem* findItem(const string& name) {
        for (auto item : items)
            if (item->getText() == name)
                return item;
        return nullptr;
    }

    ~Menu() {
        for (auto item : items)
            delete item;
    }
};
```

```
class MenuBar {
    vector<Menu*> menus;

public:
    void add(Menu* m) {
        menus.push_back(m);
    }

    void print() {
        for (auto m : menus)
            m->print();
    }

    Menu* findMenu(const string& name) {
        for (auto m : menus)
            if (m->getText() == name)
                return m;
        return nullptr;
    }

    ~MenuBar() {
        for (auto m : menus)
            delete m;
    }
};

int main() {
    MenuBar bar;

    // File menu
    Menu* file = new Menu("File");

    // Submenu of File → New
    Menu* newMenu = new Menu("New");
    newMenu->add(new MenuItem("Text", new CreateAction()));
    newMenu->add(new MenuItem("C++", new CreateAction()));

    file->add(newMenu);
    file->add(new MenuItem("Exit", new ExitAction()));

    // About menu (no action/submenu)
    Menu* about = new Menu("About");

    // Add top-level menus
    bar.add(file);
    bar.add(about);

    // Show menu
    bar.print();

    cout << "\n-- Simulating Clicks --\n";
    file->clicked();

    MenuItem* newItem = file->findItem("New");
    if (Menu* newMenuPtr = dynamic_cast<Menu*>(newItem)) {
        newMenuPtr->clicked();

        MenuItem* cppItem = newMenuPtr->findItem("C++");
        if (cppItem) cppItem->clicked();
    }

    MenuItem* exitItem = file->findItem("Exit");
    if (exitItem) exitItem->clicked();

    return 0;
}
```

SALE

3. Write a C++ application which simulates how sale discounts are applied, as follows:
   a. A *Sale* contains several *SaleItem*. Each *SaleItem* has a code and a price. **(0.5p)**
   b. The class *Discount* is abstract. Its role is to help compute the discount for a given sale. **(0.5p)**
   c. The class *ItemDiscount* computes a discount for those elements in a sale that have a given code, by discounting the item's price by the given percentage. **(0.75p)**
   d. The class SumDiscount computes a sum of discounts, as shown on the diagram. The discounts may be applied for different products. **(1p)**
   e. The main application will create a sale with 3 items. Two of the items have a discount of 10% and 15%, respectively. The application will print the total discount for the given sale. **(1p)**
   f. Take memory management into consideration and implement it correctly. **(0.25p)**




```
class SaleItem {
    int code;
    double price;

public:
    SaleItem(int c, double p) : code(c),
price(p) {}

    int getCode() const { return code; }
    double getPrice() const { return
price; }
};

class Sale {
    vector<SaleItem> items;

public:
    void addItem(const SaleItem& item) {
        items.push_back(item);
    }

    const vector<SaleItem>& getItems()
const {
        return items;
    }
};

class Discount {
public:
    virtual double calc(const Sale& s)
const = 0;
    virtual ~Discount() = default;
};

class ItemDiscount : public Discount {
    int code;
    int percent;  // e.g. 10 → 10%

public:
    ItemDiscount(int c, int p) :
code(c), percent(p) {}

    double calc(const Sale& s) const
override {
        double total = 0;
        for (const auto& item :
s.getItems()) {
            if (item.getCode() == code)
                total += item.getPrice()
* percent / 100.0;
        }
        return total;
    }
};
```

```
class SumDiscount : public Discount {
    vector<Discount*> discounts;

public:
    void add(Discount* d) {
        discounts.push_back(d);
    }

    double calc(const Sale& s) const override {
        double total = 0;
        for (auto d : discounts)
            total += d->calc(s);
        return total;
    }

    ~SumDiscount() {
        for (auto d : discounts)
            delete d;
    }
};

int main() {
    Sale sale;
    sale.addItem(SaleItem(101, 100));
    sale.addItem(SaleItem(102, 200));
    sale.addItem(SaleItem(101, 50));  // same code as
first → same discount

    SumDiscount* totalDiscount = new SumDiscount();
    totalDiscount->add(new ItemDiscount(101, 10));  // 10%
on code 101
    totalDiscount->add(new ItemDiscount(102, 15));  // 15%
on code 102

    double discountValue = totalDiscount->calc(sale);
    cout << "Total discount: " << discountValue << "\n";

    delete totalDiscount;

    return 0;
}
```